



Lecture 3 : Prefab, Spawning, Object lifetime

By Antoine McNabb-Baltar



Lecture Plan

- More Rotation
 - Prefabs
 - Creating and destroying objects
 - Pooling
 - Activating and Deactivating gameObjects,
 - Object lifespan
- Scene Management

—

Rotation



Rotation and orientation in Unity

Unity uses a left-handed coordinate system. In Unity, you can use both Euler angles and **quaternions** to represent rotations and orientation. These representations are equivalent but have different uses and limitations.

Typically, you rotate objects in your **scene** using the **Transform component**, which displays orientation as a Euler angle. However, Unity stores rotations and orientations internally as quaternions, which can be useful for more **complex motions** that might otherwise lead to gimbal lock.

Euler angles

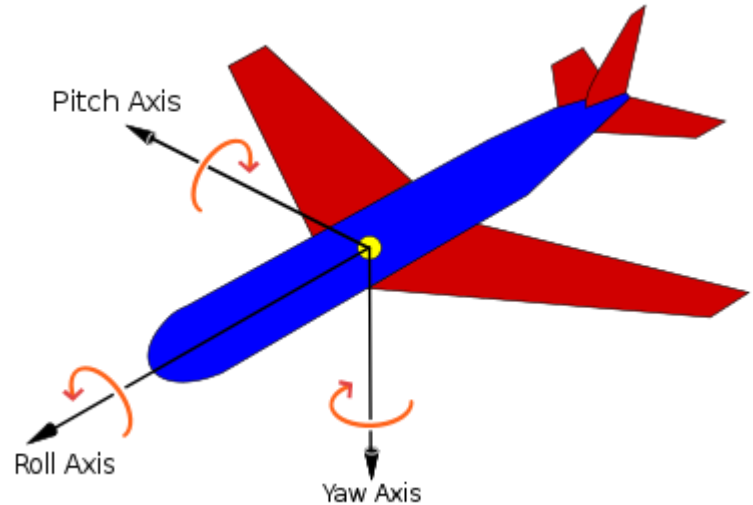
Pitch is rotation around the X axis

Roll is rotation around the Z axis

Yaw is a rotation around the Y axis

The order of rotation (usually XYZ) is important because rotating changes the object relative axis of rotation.

Ex: Tears of the Kingdom





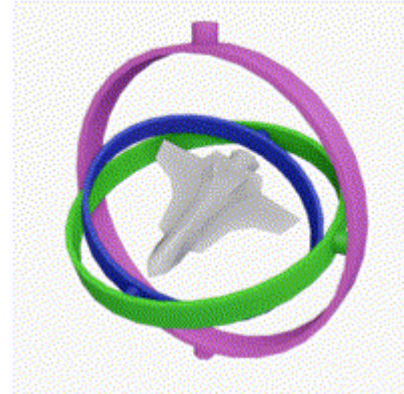
Gimbal lock

When an object in 3D space loses a degree of freedom and can only rotate within two dimensions, it's called **gimbal lock**. Gimbal lock can occur with Euler angles if two axes become parallel. If you don't convert the rotational values to Euler angles in your script, the use of quaternions should prevent gimbal lock.

If you do have problems with gimbal lock, you can avoid Euler angles if you use `Transform.RotateAround` for rotations. You can also use `Quaternion.AngleAxis` on each axis and multiply them together (quaternion multiplication applies each rotation in turn).

Gimbal lock

Gimbal lock is the loss of one degree of freedom in a three-dimensional rotation, three-gimbal mechanism that occurs when the axes of two of the three gimbals are driven into a parallel configuration, "locking" the system into rotation in a degenerate two-dimensional space.





How a Gimbal lock happens

If you rotate an object 90s degree in X you will noticed that rotation the object around the Z and Y axis creates a rotation around the same axis.

Exemple in Unity: <https://www.youtube.com/watch?v=kB7iE8Udq5g>



How to avoid Gimbal Lock

To avoid gimbal lock in Unity, **never modify an object's `transform.rotation.eulerAngles` directly and do not extract Euler vectors from existing rotations to calculate new ones**. Instead, you should calculate all rotations using `Quaternion` objects and combine them using multiplication (`*`), which natively prevents gimbal lock by operating in 4D space rather than sequentially through 3D axes



3 ways to implement rotation without gimbal lock

Incremental Rotations (e.g., Flight Sims, Free-cam, Spacecraft)

If you want to rotate an object relative to its current orientation, multiply its existing rotation by a fresh rotation step.

Important: In Unity, multiplying quaternions acts like addition, but **order matters**.

```
void Update()
{
    float pitch = Input.GetAxis("Vertical") * 50f *
Time.deltaTime;
    float yaw = Input.GetAxis("Horizontal") * 50f *
Time.deltaTime;

    // 1. Create a incremental rotation step
    Quaternion rotationStep = Quaternion.Euler(pitch, yaw,
0);

    // 2. Multiply to combine (Local Rotation)
    // New Rotation = Current Rotation * Rotation Step
    transform.rotation = transform.rotation * rotationStep;
}
```



3 ways to implement rotation without gimbal lock

2. Tracking Independent Floats (e.g., First-Person/FPS Cameras)

If you are building a standard mouse-look script, you *can* use Euler angles safely, but **only** if you store the absolute degrees in your own separate variables and construct a brand-new Quaternion every frame.

[\[1\]](#)

```
private float rotationX = 0f;  
private float rotationY = 0f;
```

```
void Update()  
{  
    rotationY += Input.GetAxis("Mouse X") * sensitivity;  
    rotationX -= Input.GetAxis("Mouse Y") * sensitivity;  
    rotationX = Mathf.Clamp(rotationX, -90f, 90f); //  
    Prevents flipping over
```

```
    // Construct a completely clean quaternion from your  
    independent variables  
    transform.rotation = Quaternion.Euler(rotationX,  
    rotationY, 0f);  
}
```

3 ways to implement rotation without gimbal lock



3. Facing Objects smoothly (e.g., Look-at behaviors)

Instead of calculating the mathematical angle to a target, let Unity's built-in methods construct the quaternion directly from raw directional data. [\[1\]](#)

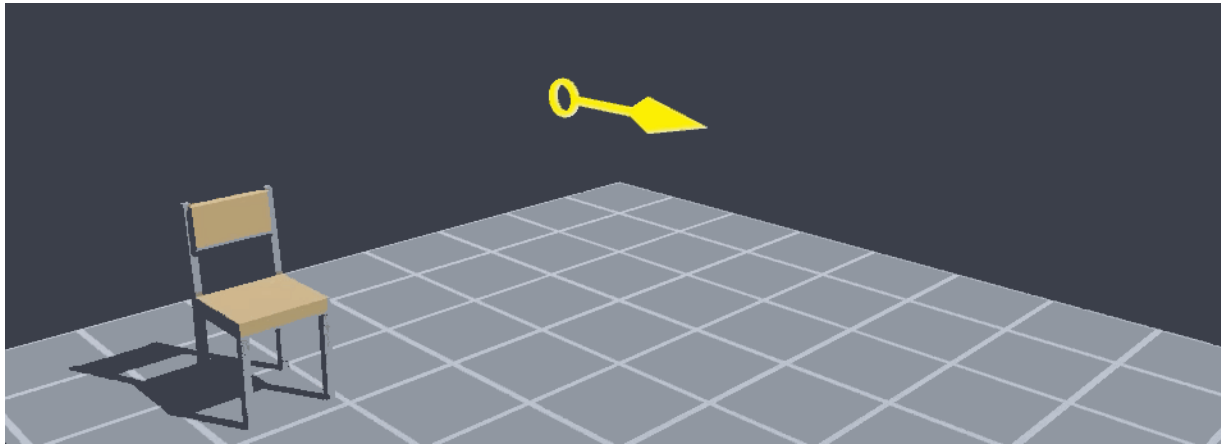
```
public Transform target;
public float speed = 5f;

void Update()
{
    Vector3 direction = target.position - transform.position;

    if (direction != Vector3.zero)
    {
        // Generate the exact target quaternion directly from a
        // direction vector
        Quaternion targetRotation =
            Quaternion.LookRotation(direction);

        // Slerp (Spherical Linear Interpolation) guarantees the
        // shortest path safely
        transform.rotation = Quaternion.Slerp(transform.rotation,
            targetRotation, speed * Time.deltaTime);
    }
}
```

How to use a Look at ?





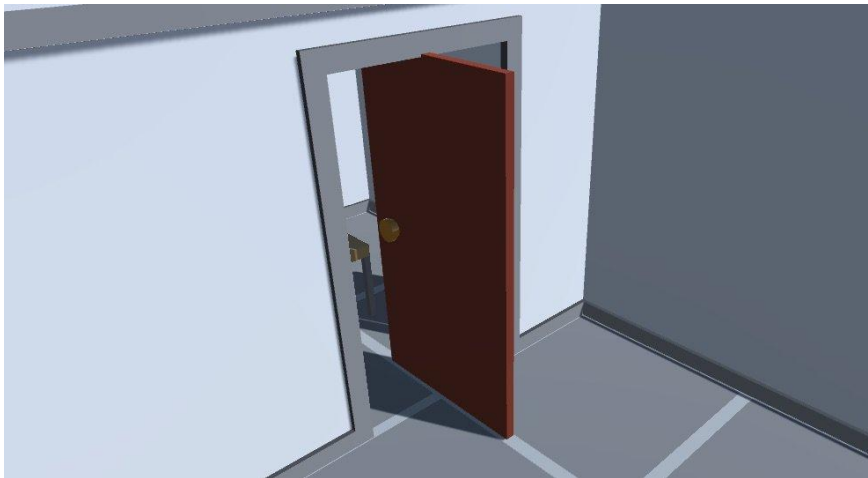
How to use Look At

Changes the rotation so that the forward vector of your gameobject is towards a **object you want** to look at:

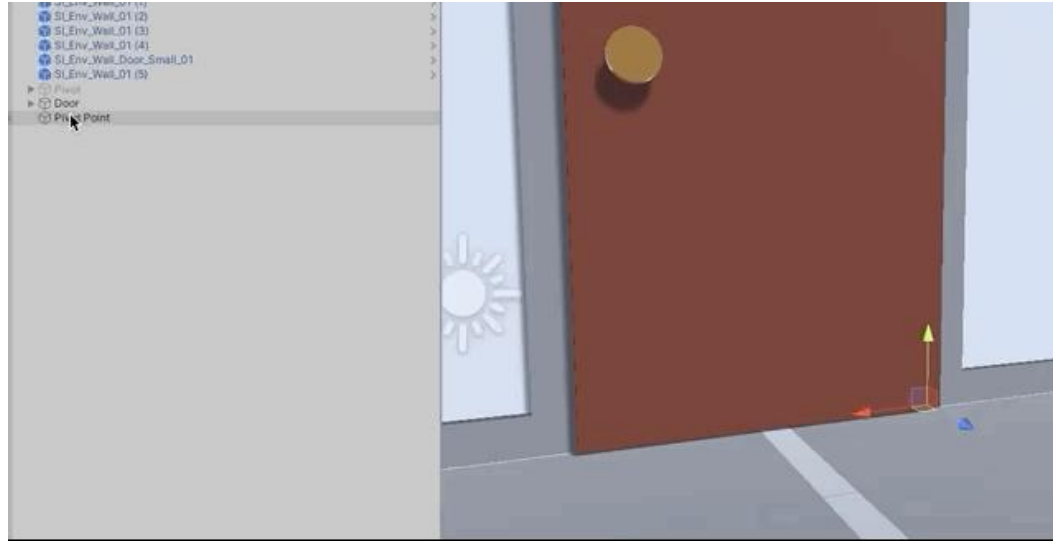
```
public Transform objectToLookAt;  
void Update()  
{  
    transform.LookAt(objectToLookAt);  
}
```

Using a pivot point as a parent game object

You can use the scene hierarchy to your advantage by rotating your parent instead of your object.



Using a pivot point as a parent game object





Pivot points and RotateAround

But you can also do it in code.

```
Vector3 pivotPoint = new Vector3 (-3.5f,0,0);  
void RotateAroundPoint()  
{  
    // Rotates around the pivot point and the Y-Axis by 90 degrees  
    transform.RotateAround(pivotPoint, Vector3.up, 90);  
}
```

**How to access reference other
scripts or object ?**

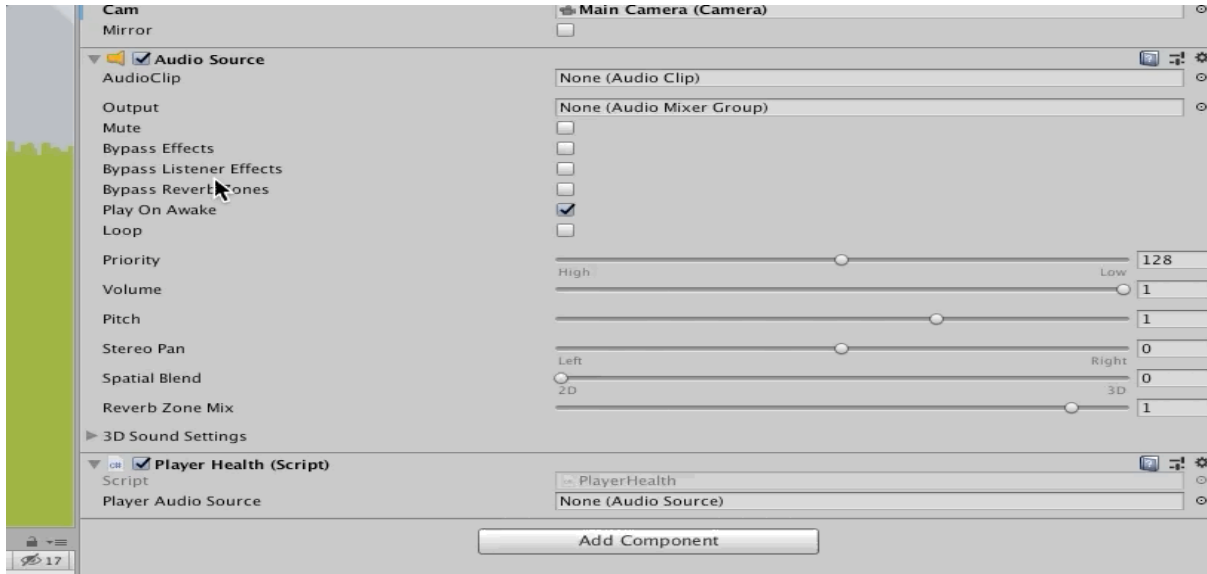


Public variables

```
public class PlayerHealth : MonoBehaviour
{
    public AudioSource playerAudioSource;
}
```

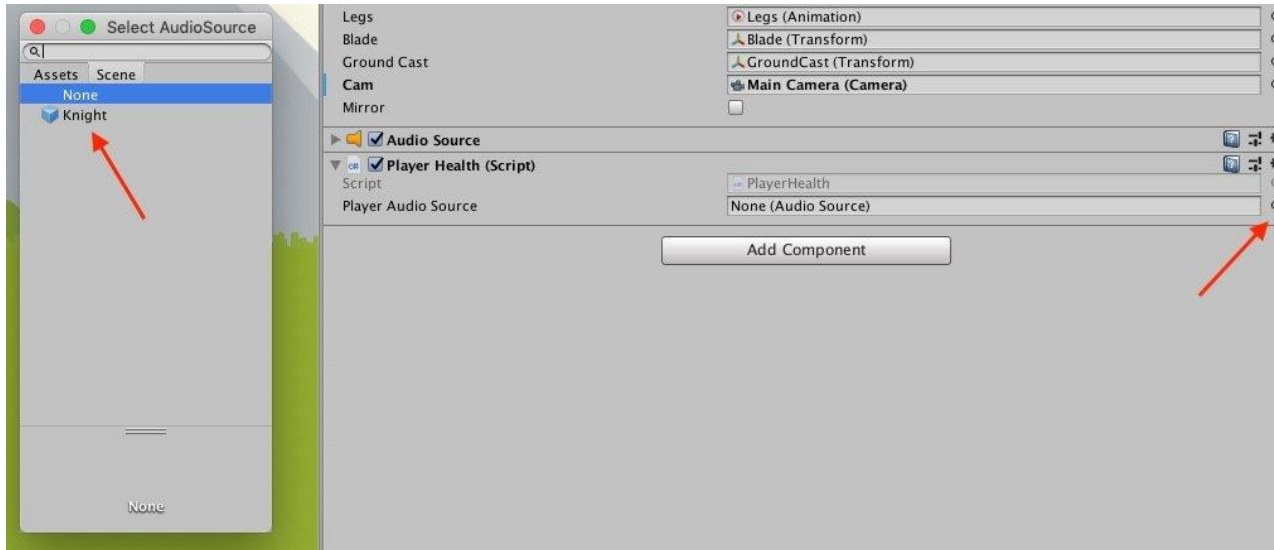
Public variables

Drag and drop the component you want to reference from the same game object.



Public variables : using the circle tool

Choose the GameObject to grab the Component (in this case AudioSource)



Public variable : Using your own mono behaviour

```
public class PlayerHealth : MonoBehaviour
{
    public AudioSource playerAudioSource;
    void PlayerHurt()
    {
        // Deal damage!
        playerAudioSource.Play();
    }
}
```

```
public class SomeOtherScript : MonoBehaviour
{
    public PlayerHealth playerHealth;
    void Start()
    {
        playerHealth.playerAudioSource.Play();
    }
}
```





Using Get Component

Get Component is a method for finding a specific type of component.

It allows you to search for a specified type of component on one or a number of game objects and get a reference to it.

This is especially useful for getting references to components that are on the same object as the calling script, but it can also be used to search for components on other game objects as well.



Using Get Component

Using

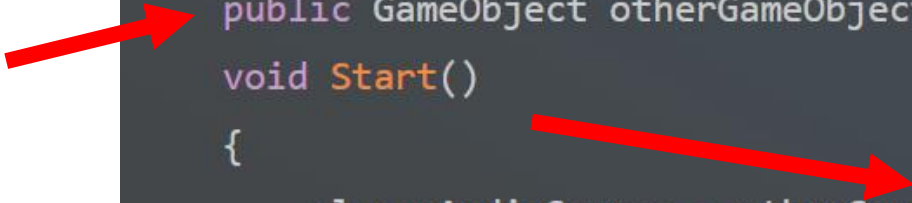
```
GetComponent<AudioSource>();
```

Will give you a reference to the first AudioSource component on your GameObject

.

Get Component from a different object

```
public class PlayerHealth : MonoBehaviour
{
    public AudioSource playerAudioSource;
    public GameObject otherGameObject;
    void Start()
    {
        playerAudioSource = otherGameObject.GetComponent<AudioSource>()
    }
}
```





Get Component in Children

Get Component in Children works in the same way as **Get Component** except, instead of only checking the specified game object, it checks any child objects as well.

```
public class PlayerHealth : MonoBehaviour
{
    AudioSource playerAudioSource;
    void Start()
    {
        playerAudioSource = GetComponentInChildren<AudioSource>();
    }
}
```



Get Component in Children

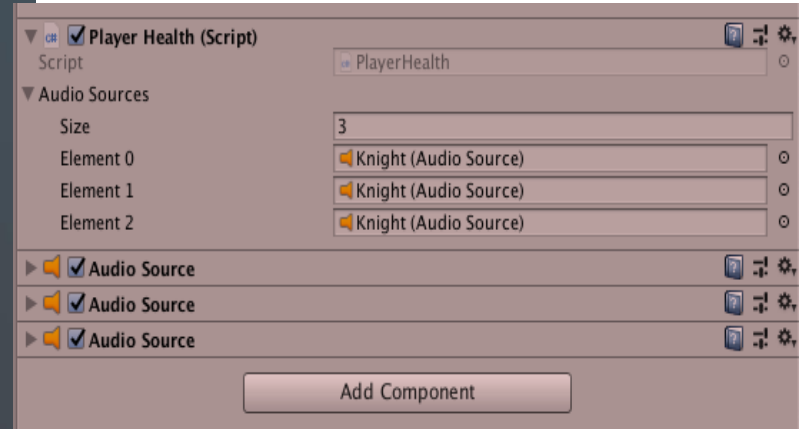
Get Component in Parent checks both the game object it's called on and the object's parent.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
public class PlayerHealth : MonoBehaviour
{
    public AudioSource playerAudioSource;
    void Start()
    {
        playerAudioSource = GetComponentInParent<AudioSource>();
    }
}
```

How to get the second component on a game object (managing multiple components)

Using arrays and filling it with `GetComponents<>`.

```
public class PlayerHealth : MonoBehaviour
{
    public AudioSource[] audioSources;
    public AudioSource playerAudioSource;
    void Start()
    {
        audioSources = GetComponents<AudioSource>();
        playerAudioSource = audioSources[1];
    }
}
```





Finding different objects in the Scene

This will return a game object that matches an exact string.

In this case, "Player".

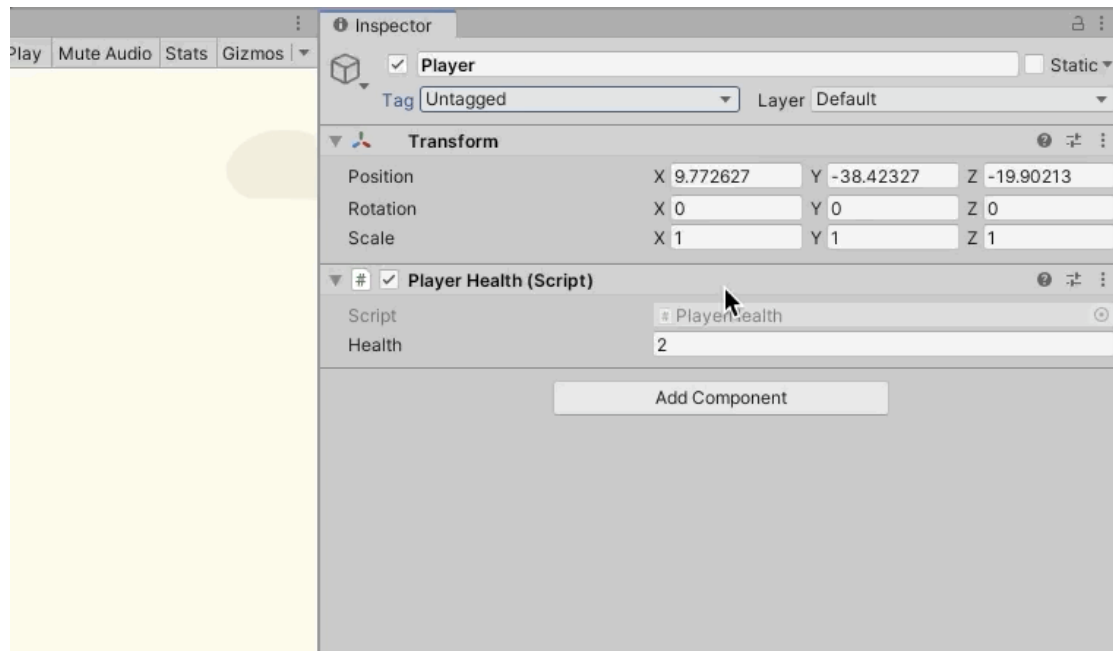
```
public class FindObject : MonoBehaviour
{
    public GameObject player;
    void Start()
    {
        player = GameObject.Find("Player");
    }
}
```



How to find an object by its tag in Unity

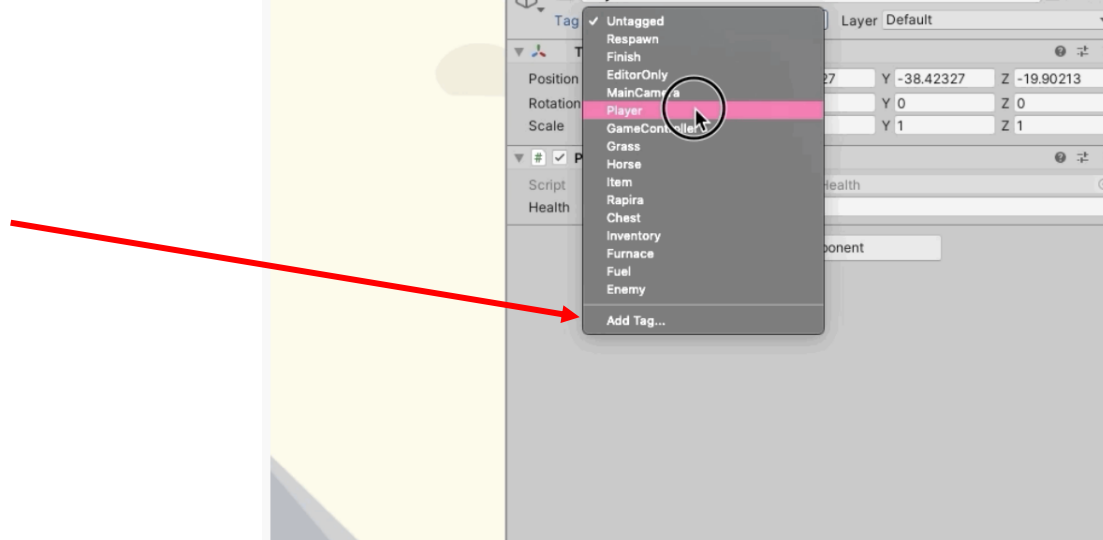
```
private void OnTriggerEnter(Collider other)
{
    if (other.tag == "Player")
    {
        // Do Something
    }
}
```

Setting a Tag



Adding new tags

In the Tag drop down at the very bottom is a **Add Tag Option**.





How to get multiple objects with tags

Just like Find with Tag, Find Objects with Tag finds objects in the Scene with a certain tag attached, adding them all to an array.

```
public class FindObject : MonoBehaviour
{
    public GameObject[] enemies;
    void Start()
    {
        enemies = GameObject.FindGameObjectsWithTag("Enemy");
    }
}
```



Find Objects of Type

Find Objects of Type can be useful for finding game objects that share the same script or component.

For example you could use it to find all of the audio sources in a Scene .

```
public class FindObject : MonoBehaviour
{
    public AudioSource[] audioSourcesInScene;
    void Start()
    {
        audioSourcesInScene = FindObjectsOfType<AudioSource>();
        foreach(AudioSource audioSource in audioSourcesInScene)
        {
            audioSource.mute = true;
        }
    }
}
```



Find Objects of Type

You can also references custom scripts

```
public class FindObject : MonoBehaviour
{
    public PlayerHealth[] objectsWithHealth;
    void Start()
    {
        objectsWithHealth = FindObjectsOfType<PlayerHealth>();
    }
}
```

Copy



Global variables in Unity

Global variables in Unity generally refer to a variable that has a single value, a single point of reference and that is accessible from any script.

There are several methods for creating **global variables in Unity**.

The simplest method for creating a global variable is to use a **static** variable.



Global variables Method 1: Static variables

```
public static class PlayerHealth
{
    public static float health;
    public static float armour;
}
```

is to use a static variable.



Global variables Method 1: Static variables

```
Float hp = PlayerHealth.health;
```



Global variables: Using statics to create a Singleton game manager

One method of using static variables to manage global references and values is to create a **Singleton** game manager.

A Singleton uses a **single, static**, point of entry via a **static instance reference**.

That way other scripts can access the Singleton, and its variables, without getting a reference to it first.



Global variables: Using statics to create a Singleton game manager

The **Game Manager** script then keeps a static reference to itself, which is accessible to other scripts.

This script **needs** to be added to **an object in the Scene** in order to work, so the class itself isn't **static**, just its **instance reference**.



Global variables : Singleton

```
public class GameManager : MonoBehaviour
{
    public static GameManager Instance { get; private set; }
    public float playerHealth=100;
    public float gameTime = 90;
    private void Awake()
    {
        Instance = this;
    }
}
```



Accessing the singleton variables or methods

```
void Start()
{
    Debug.Log(GameManager.Instance.playerHealth);
    Debug.Log(GameManager.Instance.gameTime);
}
```

Creating and Destroying object



Prefabs

Prefabs are a **special type of component that allows fully configured GameObjects to be saved in the Project for reuse**. These assets can then be shared between scenes, or even other projects without having to be configured again. This is quite useful for objects that will be used many times, such as platforms.



Why use prefabs ?

The prefab assets **allow you to store an asset with all its properties inside the prefab**, and the prefab acts basically as a template that you use to create new instances of the same object in the scene.

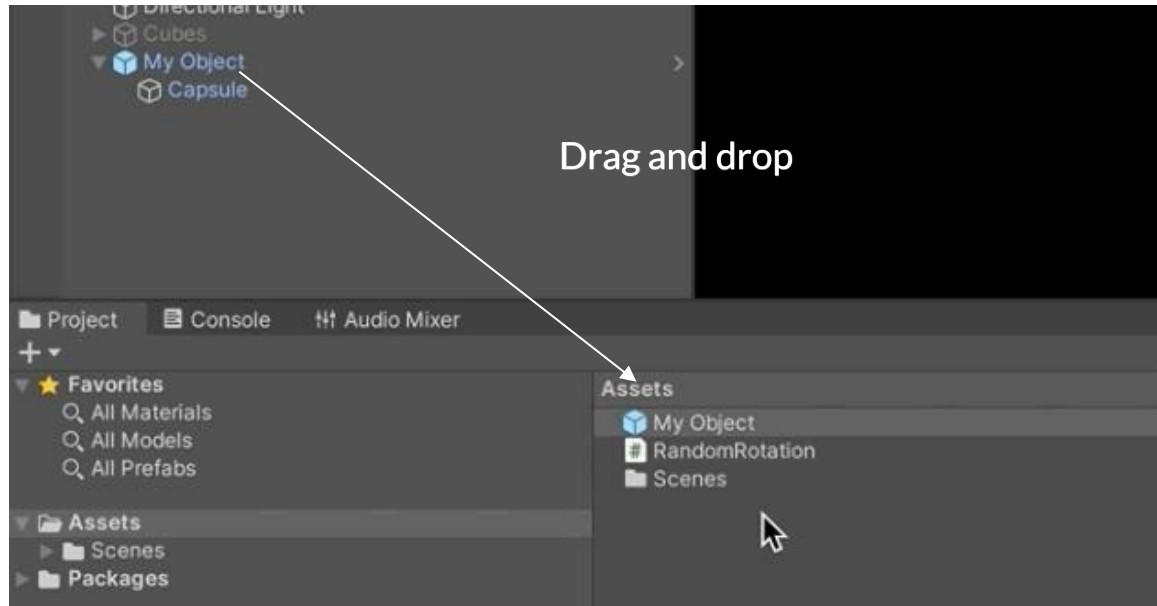


How to create a prefab

The simplest way to do this is to **build your object inside your scene**. Then, once you're done, **drag** the object to your project's assets to create a **prefab** of it. Your new prefab will appear in your project's assets, while the object in your scene **will turn blue, highlighting** the connection between the prefab template and the object in the hierarchy.

You'll then be able to **delete** the object from your **scene** and use the **prefab** in the Instantiate function.

How to create a prefab





IMPORTANT : Difference between Game Object and Prefabs

1. Game Object:

- **Definition:** A Game Object is an instance of an object in your scene. It can represent anything in the game, such as characters, props, lights, cameras, or other entities.
- **Instance:** Exists directly in a scene and has components that define its behavior (e.g., Transform, Rigidbody, Colliders, etc.).
- **Editable:** You can modify Game Objects directly in the scene and save those changes in the scene file.
- **Examples:** A tree, an NPC, or a camera in your game scene is a Game Object.

2. Prefab:

- **Definition:** A Prefab is a template or blueprint for a Game Object or a group of Game Objects. It allows you to store reusable objects in a project, which can be instantiated multiple times.
- **Reusable:** Prefabs are designed to be instantiated multiple times, which means you can use the same Prefab to spawn multiple identical objects across various scenes or within the same scene.
- **Maintained Changes:** When you modify the original Prefab, those changes can be applied to all its instances. However, each instance can also have its own specific changes or overrides.
- **Examples:** A car model stored as a Prefab that can be instantiated at runtime as multiple car objects.



IMPORTANT : Difference between Game Object and Prefab

Key Differences:

- **Scene Presence:** A Game Object exists in a scene, whereas a Prefab is stored in the project folder and can be instantiated into scenes as needed.
- **Modification:** Game Objects in the scene are directly editable and can be customized. Prefabs serve as a master copy that can be used to create multiple instances of Game Objects.
- **Use Case:** Use Game Objects to directly interact with elements in your scene, and Prefabs to manage reusable assets that can be spawned dynamically or shared across different scenes.



IMPORTANT : Difference between Game Object and Prefabs

Dragging **Gameobject** from your **Scene Hierachy** will use the **Game Object** . If you delete the object your referenced in your script the code will stop working .

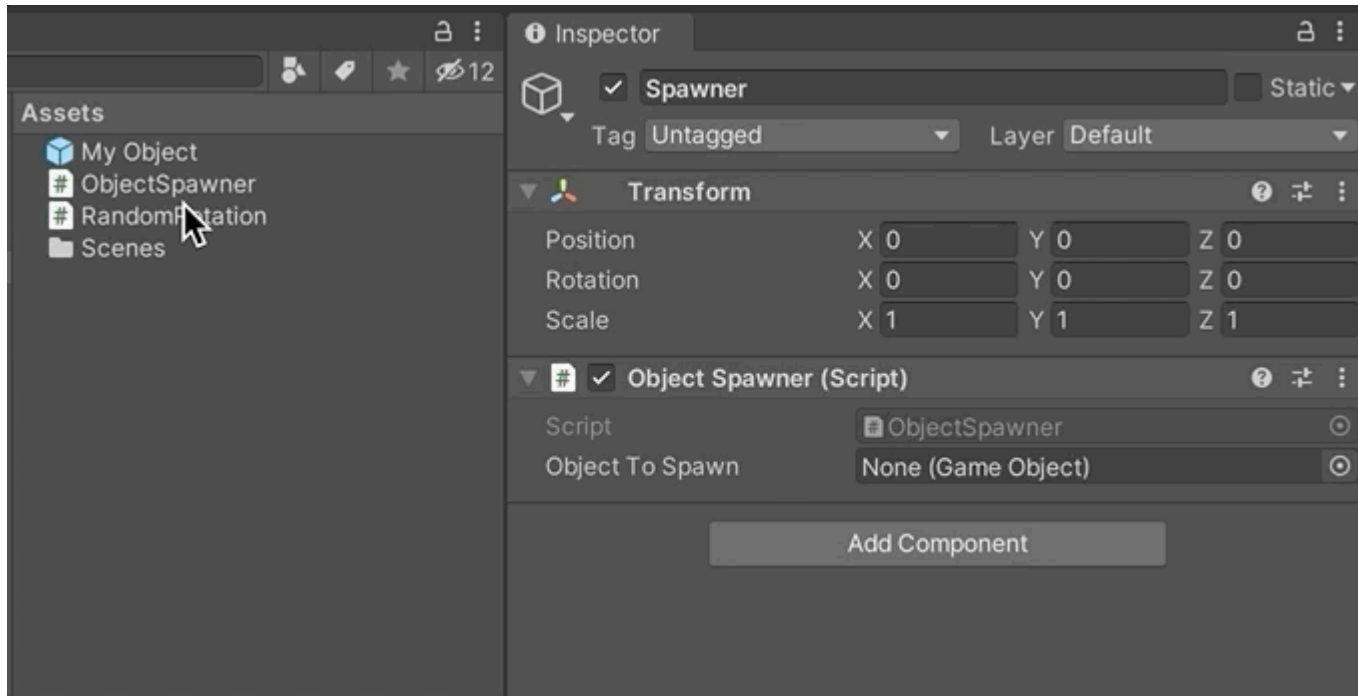
Dragging a Prefab from your **Asset Folder** will use the Prefab. Destroying a prefab during gameplay wont affect your code.



Spawning new GameObject via a Prefab

```
public GameObject objectToSpawn;  
void Start()  
{  
    Instantiate(objectToSpawn);  
}
```

Spawning a new GameObject via Prefab





Spawning a new GameObject with Prefab

To keep a reference to the spawn GameObject :

```
void SpawnObject()  
{  
    GameObject newObject = Instantiate(objectToSpawn);  
}
```



How to set the rotation and position of a new object

While the basic overload method of `Instantiate` only requires a game object reference to work, chances are, you'll want to pass in more information than that when creating a new object.

By default, an **instantiated** object inherits the **position and rotation** of the object it's **cloned** from. Meaning that, if **you clone** an object in the scene, the new object will be created in the **same position as the original**.



How to set the rotation and position of a new object

The method to use to define Instantiate a GameObject with a specific location and a default rotation is:

```
Instantiate(objectToSpawn, transform.position, Quaternion.identity);
```



Spawn a new instance as child of an object.

```
// Creates a child object but uses custom position and rotation values
```

```
Instantiate(objectToSpawn, transform.position, transform.rotation, transform);
```



Prefab

Spawned Position

Spawn Rotation

Parent Object



Spawn multiples new object

```
Vector3 position = firstPosition;
for (int i = 0; i < 5; i++)
{
    Instantiate(objectToCreate, position, Quaternion.identity);
    position.x += gap;
}
```



Spawn object at intervals

```
public GameObject objectToCreate;
    public float interval = 5;
    float timer;
    void Update()
    {
        timer += Time.deltaTime;
        if (timer >= interval)
        {
            Instantiate(objectToCreate);
            timer -= interval;
        }
    }
}
```

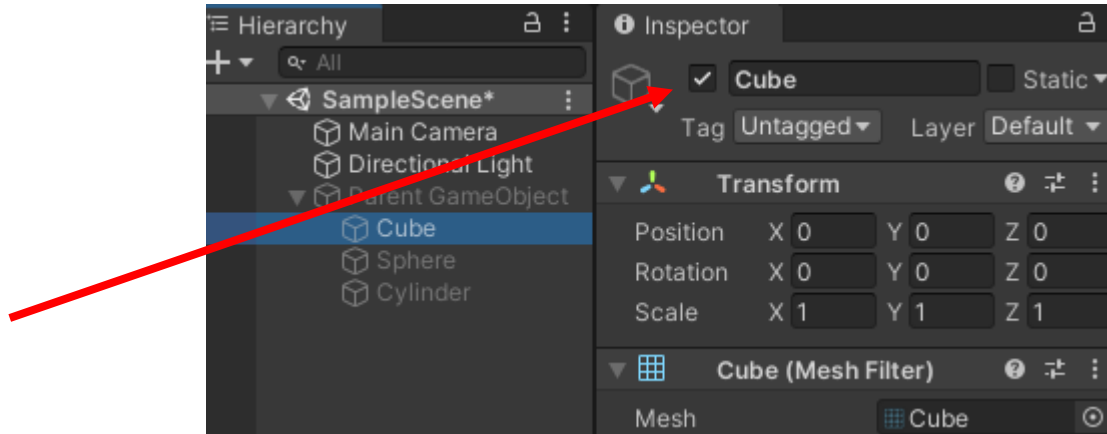


How to destroy a GameObject?

```
Destroy(Object objectToDestroy);
```

Disabling GameObject

To temporarily remove a **GameObject** from your **scene** you can mark the **GameObject** as inactive.





How to disable in code ?

When disabling an object using the Set Active function, the game object will report itself to be disabled immediately.

```
gameObject.SetActive(false);
```



Destroying GameObject

To destroy an object in Unity, simply call the **Destroy** function, passing in a reference to the object that you want to destroy.

```
Destroy(Object objectToDestroy);
```



Delayed destroying of GameObject

It's also possible to pass in a float delay value when calling the destroy method.

This will cause the destroy function to trigger in a number of seconds, which is specified by the float you pass in, instead of right away.

```
// Destroys the object in 4 seconds  
Destroy(gameObject, 4);
```



When are destroyed objects removed in Unity?

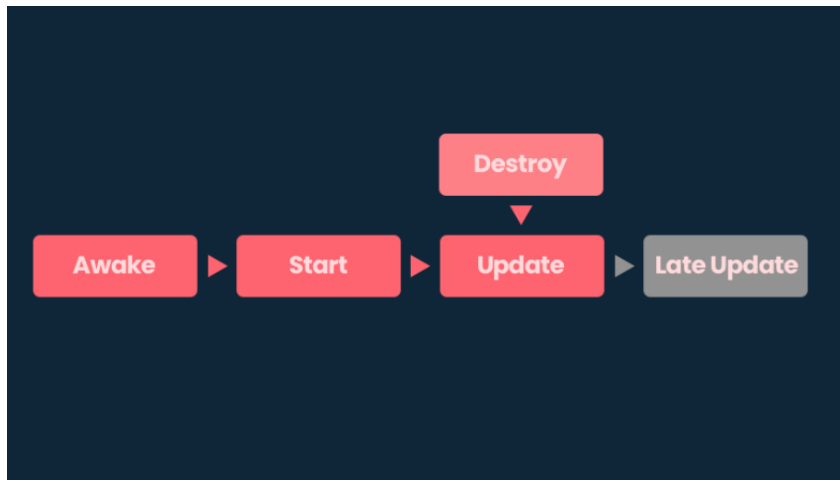
It may surprise you to know that, when you destroy an object, it **isn't removed from the scene immediately**.

Instead, it's marked for deletion and removed after the current round of event functions that it's destroyed in.

For example, it's possible to call the destroy function on an object and then, **immediately** after “destroying” it, access the object in some way, without causing an error.

When are destroyed objects removed in Unity?

So, for example, if an object is destroyed in `Update`, it will still exist for the rest of the `Update` function in that script, and in any other `Update` functions on other scripts during that frame.





Destroying vs Deactivating

Destroying (**Destroy**)

- **Removes completely:** Deletes the object and its components from the scene and memory.
- **Frees memory:** Clears RAM, though garbage collection can cause performance hiccups later.
- **One-way trip:** You cannot turn the object back on; you must spawn a new one using **Instantiate**.
- **Best used for:** Loot pickups, dead unique enemies, or objects you will never need again.

Deactivating (**SetActive(false)**)

- **Stops activity:** Halts rendering, scripts, and physics calculations immediately.
- **Keeps memory:** The object stays loaded in RAM, ready to turn back on instantly.
- **Fast performance:** Avoids lag spikes caused by creation and deletion (ideal for **object pooling**).
- **Best used for:** Bullets, UI menus, or repeating enemies that you want to reuse



Destroying vs Deactivating

On Disable and **On Destroy** can be used to decommission an object before disabling it, either before turning it off or before it's destroyed.

They are call back to destroying an object or deactivating it but they do not happen at the same time.



Destroying vs Deactivating

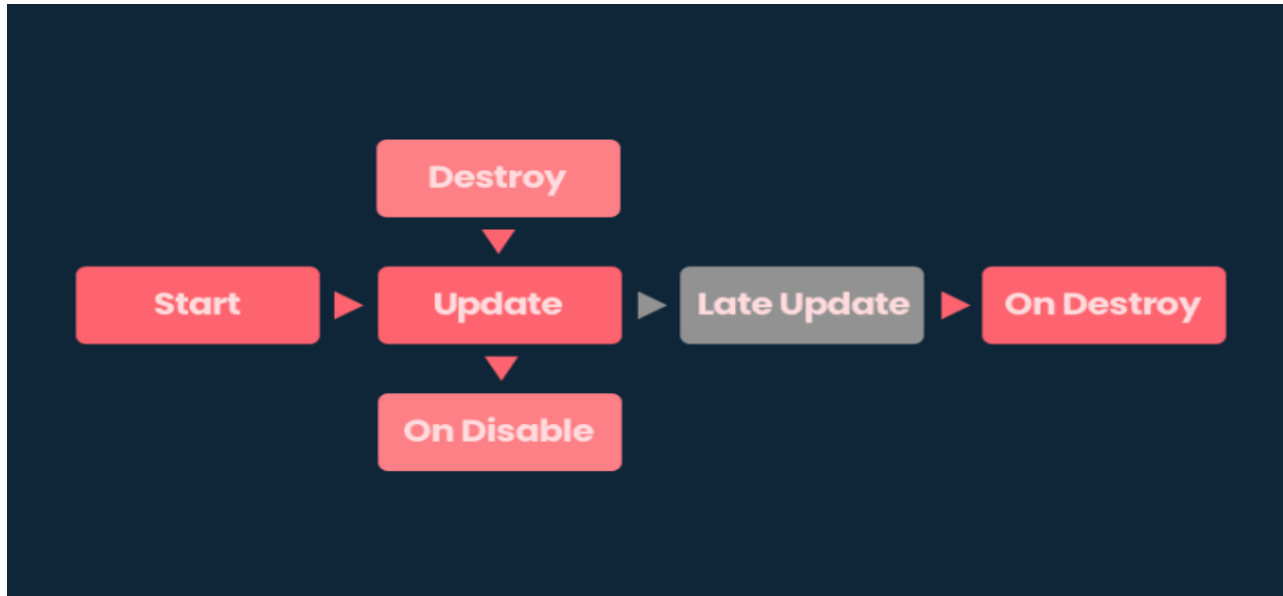
```
void OnDisable()  
{  
    // Called when an object is turned off  
}  
void OnDestroy()  
{  
    // Called when an object is destroyed  
}
```



Destroy vs Deactivating

When an object is destroyed, **On Disable** and **On Destroy** will both be called in its scripts, starting with **On Disable**, which is called at the same time as the initial **Destroy** function, and ending with **On Destroy** at the end of the frame.

Destroy vs Deactivating





Destroy vs Deactivating

However, while both destroying an object and simply turning it off will cause the **On Disable function to be called**, there is a slight difference between how the two methods work.

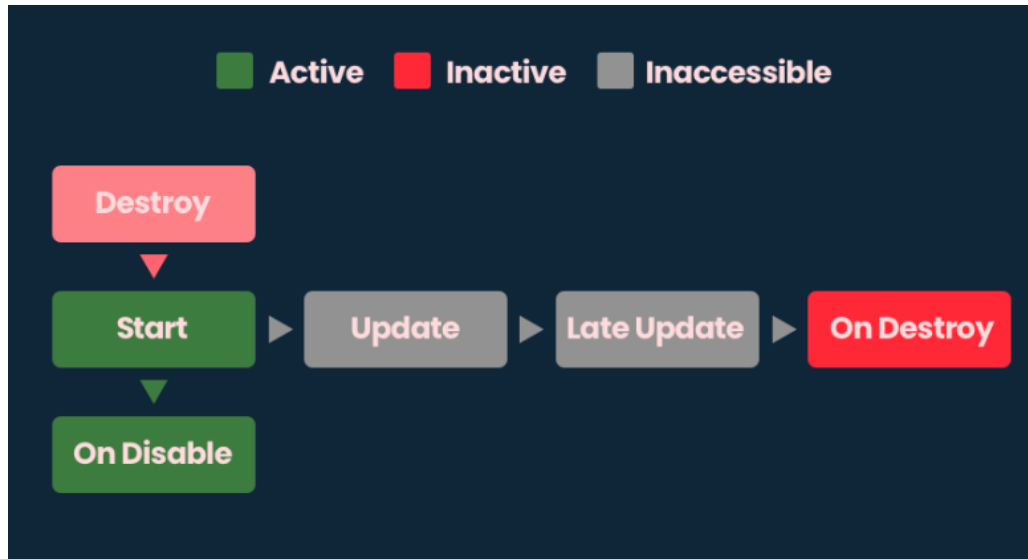
When disabling an object using the Set Active function, the game object will **report itself to be disabled immediately**.

What's more, even though it's turned off, other scripts with a reference to the object **will be able to check if it is active** or not for as long as the object still exists.

Destroy vs Deactivating



Destroy vs Deactivating





Should you destroy or disable an object?

Generally speaking, it's usually better to turn an object off than to destroy it, if you can.

Turning objects off instead of destroying them is the basic idea behind **Object Pooling**, which allows you to reuse objects over and over without having to recreate them, saving performance.



Should you destroy or disable an object ?

What's more, because disabled objects still, technically, exist, turning an object off can sometimes be easier to manage than destroying it.

For example, if a script in your scene still holds a reference to a disabled object, it's possible to check if that object is still active or not, by checking the **Active Self** or **Active in Hierarchy** properties.



How to know if an object is active ?

```
bool active = gameObject.activeSelf;  
bool accessible = gameObject.activeInHierarchy;
```

Active Self will return true or false depending on if the object is, itself, turned on or off, while **Active in Hierarchy** only returns true if the object is active *and* accessible, as an active object can still be inaccessible if its parent is turned off.



Removing scripts from a GameObject using Destroy

```
Destroy(audioSource); // Removes the referenced component  
Destroy(this); // Removes this script
```

Disabled components can't be used, and will usually return a warning, not an error, if you try to do something with them.

Removed components, however, are missing and trying to access them in any way will typically result in an error.



Disabling scripts

To disable a script you need to set the Enabled value to false

```
GetComponent<MyComponent>().enabled = false;
```

Disabling a script means that its event functions, such as **Update** and **Fixed Update**, won't be called.

But that doesn't mean that other functions in the script can't be used even when it's turned off.

**Reusing instead of always destroying
and creating**



An Object Pool ?

Object pooling is the process of **reusing an object instead of repeatedly destroying and instantiating** it, which can create garbage and reduce performance.

There are many different ways to create an object **pool** but it typically involves maintaining a data collection, such as an array or a list, that's linked to a certain type of object, such as a **projectile** in your game.

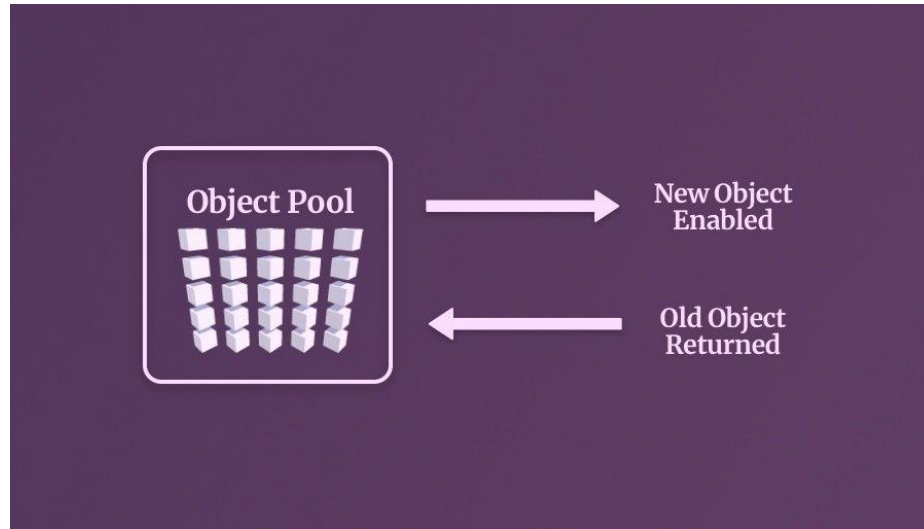


Object pooling

Old objects are added to the list when they're no longer needed, **where they're turned off or reset instead of destroyed.**

Then, when a new object of that type is required, it can be enabled from the list, **the object pool,** where it will be turned back on, instead of being instantiated, **saving performance.**

How Object Pooling works ?





When Do You Actually Need Pooling?

You should consider pooling when:

- You **instantiate and destroy gameobjects in rapid succession**, e.g. weapon bullets.
- You **frequently allocate and deallocate objects stored in the heap** (instead of reusing them). That includes C# collections.



When Do You Actually Need Pooling?

These operations cause lots of allocations and therefore:

- A waste of CPU cycles for instantiation and destroy operations (or new/dispose).
- Premature garbage collections that end in these gameplay freezes your players hate
- Memory fragmentation that make it hard to find free contiguous memory regions



Why not use object pooling ?

- **Your items might be dirty.**

Since they were used in the past, you might have left them in an undesirable state, such as **bullets with some red juice on them**. That means, you need to spend some CPU cycles to clean your items before using them: the **reset** operation.

- **You reserve memory that you might actually not need.**

If you pool thousands of bullets but all your player wanted to do is to look at the landscape, then you **wasted memory**.

- **It adds complexity to your codebase.**

You need to manage the lifecycle of your pools. This not only adds CPU cycles but **brain cycles** due to processing a larger codebas.



How to use Object Pools ?

We first have to include a library called Pool.

```
using UnityEngine.Pool;
```

You'll then be able to declare a new pool of a particular type, such as a game object:

```
ObjectPool objectPool = new ObjectPool<GameObject>();
```



How to use Object Pools

When you do this, you'll need to pass a reference to up to four functions that **will be called when** different requests are made to the object pool.

Create – Which is called when an object is first instantiated

Take – Which is called when an object is taken from the pool

Release – Which is called when an object is released back to the pool

Destroy – Which gets called when an object is destroyed instead of being returned

How to use object pools

You need to define 4 functions and reference them in the object pool creation like so :

```
objectPool = new ObjectPool<GameObject>(OnObjectCreate,  
OnTake, OnRelease, OnObjectDestroy);
```

```
GameObject OnObjectCreate()  
{  
    GameObject newObject = Instantiate(objectPrefab);  
    newObject.AddComponent<PoolObject>().myPool = objectPool;  
    return newObject;  
}  
  
void OnTake(GameObject poolObject)  
{  
    poolObject.SetActive(true);  
}  
  
void OnRelease(GameObject poolObject)  
{  
    poolObject.SetActive(false);  
}  
  
void OnObjectDestroy(GameObject poolObject)  
{  
    Destroy(poolObject);  
}
```

Delayed function calls



Using the invoke function

The purpose of the **Invoke** function is to call a method with a delay.

It works by passing in a **String** parameter, which is the name of the function that you want to call, and a **Float** delay, which is how long you want to wait before the function is triggered.



Using an Invoke

```
void Start()
{
    Invoke("MyFunction", 3);
}

void MyFunction()
{
    Debug.Log("Hello!");
}
```



Using an Invoke on another script

```
public OtherScript otherScript;  
void Start()  
{  
    otherScript.Invoke("MyFunction", 5);  
}
```



How to trigger a function regularly

Invoke Repeating works in the same way as **Invoke**, which uses a **String** to identify a function by name, and a **Float** to specify how long to wait before triggering it.

The difference is that **Invoke Repeating** will then, after the initial delay, repeat the function at a set interval.



Invoke Repeating

```
void Start()
{
    // Wait for 3 seconds then trigger My Function every 0.5 s
    InvokeRepeating("MyFunction", 3, 0.5f);
}

void MyFunction()
{
    // Do something!
}
```



How to cancel an already sent Invoke

You can cancel an already triggered Invoke function, before the method is executed, by using **Cancel Invoke**

Use this syntax for a specific function .:

```
CancelInvoke("MyFunction");
```

Or using this to stop your previously invoked function in this script , regardless of their name

```
CancelInvoke();
```



Drawbacks of Invoke

- The function is triggered by a string, so it's possible to break the connection if its name is ever changed.
- A method triggered using Invoke can't accept parameters.

Function with pauses (Coroutines)



What is a coroutine

A coroutine is a function that can be run over a number of frames, rather than all executed during a single frame.

Coroutine allows you to **pause** a function and tell it to **wait for a condition or action** to occur before continuing. This allows you to **split** up its functionality into a number of steps that can be executed in order.



The yield keyword in Coroutines

The primary purpose of the **yield** keyword in a Unity coroutine is to temporarily pause (suspend) the function's execution and hand control back to the main Unity game loop. This allows a function to **spread its work** across multiple frames or wait for a specific event without freezing the entire game

Yield options



What you write	What it tells Unity to do	Best used for
<code>yield return null;</code>	Pause and resume on the very next frame .	Smooth animations, gradual movement, or breaking up huge calculations.
<code>yield return new WaitForSeconds(2.0f);</code>	Pause for a specific duration of game time . Affected by <code>Time.timeScale</code> (will pause if the game is paused).	Cooldown timers, spawning enemy waves, or delayed triggers.
<code>yield return new WaitForSecondsRealtime(2.0f);</code>	Pause for a specific duration of real-world time (ignores game pause status).	UI animations or menu logic that must keep moving while the game is paused.
<code>yield return new WaitForFixedUpdate();</code>	Pause until the next physics update cycle.	Synchronizing script logic safely with rigidbodies and physics calculations.

Yield options



<pre>yield return StartCoroutine(OtherMethod());</pre>	Pause the current coroutine until a nested coroutine completely finishes.	Creating sequential events , like a cutscene where Action A must finish before Action B begins.
<pre>yield return webRequest.SendWebRequest();</pre>	Pause execution until an asynchronous operation (like loading a scene or downloading data) finishes.	Loading screens or fetching database information without freezing gameplay.
<pre>yield break;</pre>	Stop the coroutine early and exit completely .	Stopping a repeating behavior ahead of time if a specific condition is met (e.g., player dies mid-cutscene).



How to use Coroutines ?

```
void Start()
{
    StartCoroutine(WaitForFunction());
}

IEnumerator WaitForFunction()
{
    yield return new WaitForSeconds(3);
    Debug.Log("Hello!");
}
```

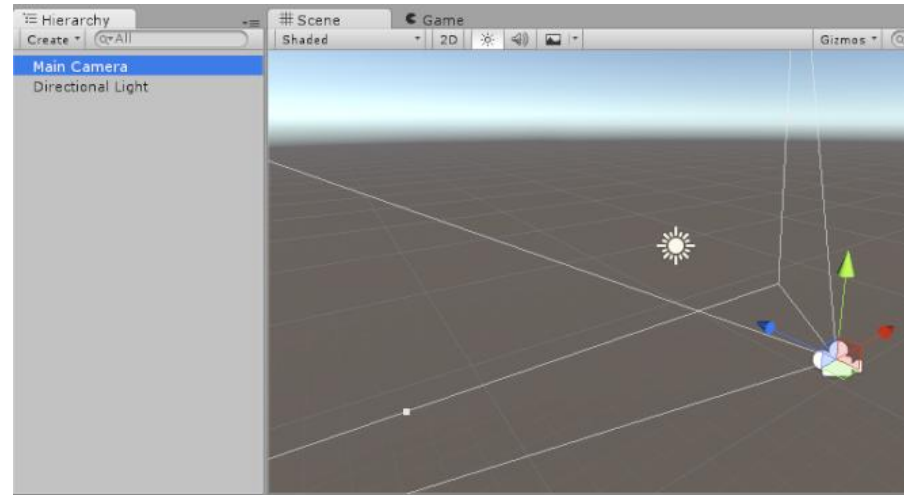
Multiple scene

Scenes

Scenes are where you work with content in Unity.

They are assets that contain all or part of a game or application. For example, you might build a simple game in a single **scene**, while for a more complex game, you might use one scene per level, each with its own environments, characters, obstacles, decorations, and **UI**.

You can create any number of scenes in a project. When you create a new project and open it for the first time, Unity opens a sample scene that contains only a **Camera** and a **Light**.





Scene Templates

Unity uses scene templates to create new scenes. Scene templates are assets that are stored in a project.

They are similar to scenes, but are designed to be copied rather than used directly.

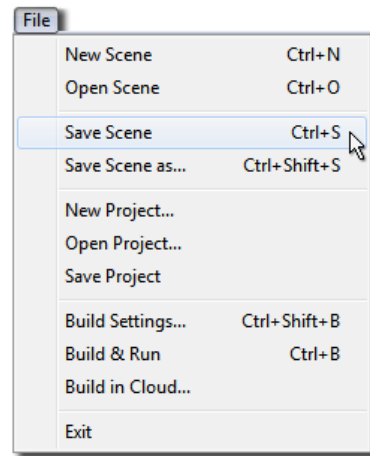
To create new scenes, Unity copies **scene** templates. Think of a scene template as a pre-configured scene that contains all of the content you want to start with.

For example, the default Basic template usually contains a **Camera** and a light.

Saving a scene

Unity stores lots of different types of information about your project, and some of these are saved in different ways to others. This means that *when* your work is saved depends on what kind of changes you are making.

Saving changes to the current scene ("Save Scene")





Saving your scene

Scene changes include modifications to any objects in the Hierarchy. For example, adding, moving or deleting Game Objects, changing parameters of hierarchy Game Objects in the inspector.

To save changes to the scene, select Save Scene from the file menu, or hit **Ctrl/Cmd + S**. This saves current changes to the scene *and* Does a “**Save Project**” (below).

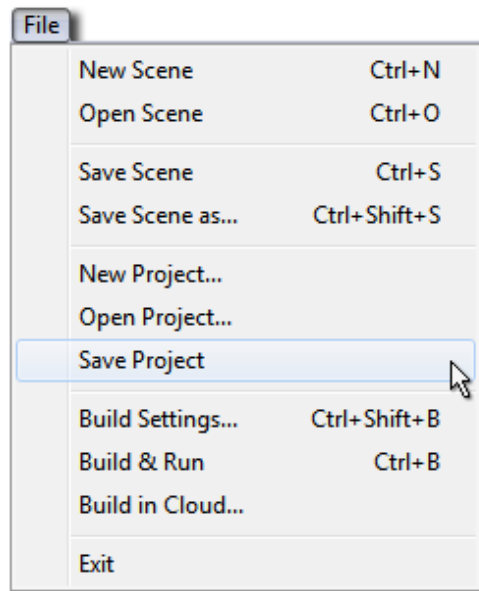
This means that when you do a “Save Scene”, *everything* is saved.

Saving project-wide changes (“Save Project”)

Some changes that you can make in Unity are not scene-specific, they are *project-wide*. These settings can be saved independently of the scene changes, by selecting “Save Project” from the file menu.

Using “Save Project” does not save changes to your Scene, only the project-wide changes are saved. You may want to, for instance, save your project but not changes to your scene if you have used a temporary scene to make some changes to a prefab.

The project-wide changes which are saved when you “Save Project” include:

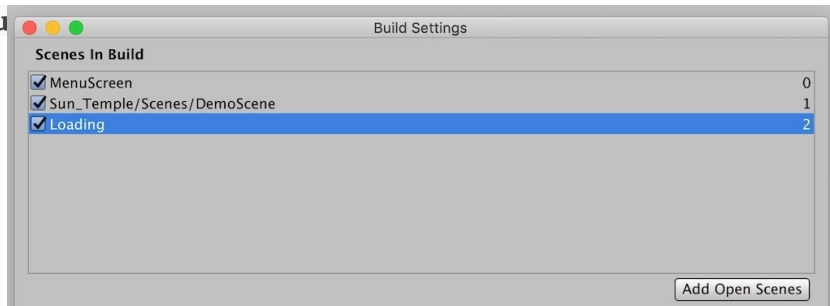


How to load a new Scene in Unity

To load any Scene from a script in Unity, first, you'll need to make sure that the Scene you want to load is included in the build.

Otherwise, if the Scene hasn't been added to the Build Settings, or the Asset Bundle hasn't been loaded, none of this will work and the Scene won't load.

To check, select **File > Build Settings...** to open the Build Settings window. You can then add Scenes from the Project View.





How to load a Scene in Unity

Next, you'll need to add the **Scene Management namespace** to any script that you want to load Scenes from.

Simply add **“using UnityEngine.SceneManagement;”** with the other using directives.



Loading a scene : Using Scene Management

This will allow you to use functions from the **SceneManager class**.

Finally, to load a new Scene, call the **Load Scene** function, passing in the name of the Scene you'd like to load.

Like this :

```
SceneManager.LoadScene("DemoScene");
```

```
using System.Collections;  
using System.Collections.Generic;  
using UnityEngine;  
  
using UnityEngine.SceneManagement;
```



Loading scene with an Index.

Alternatively, you can use the Scene's index instead.

Like this: `SceneManager.LoadScene(1);`

If you don't know the index of the Scene you want to load, you can look it up by opening the Build Settings. You can also reorder Scenes to change their index number.

Loading by a Scene's index can be especially useful if you want to load Scenes in order; For example, if you want the player to proceed to the next Scene in the list at the end of each level.



How to load the next Scene in Unity

To load Scenes in order, i.e. load whatever Scene is next on the list after the current

Scene, all you need to do is get the current Scene index and add 1.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;
public class LoadScene: MonoBehaviour
{
    public void LoadNextScene()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex + 1);
    }
}
```



The two types of loading

There are two different methods available for Loading a scene.

Load Scene and **Load Scene Async**.



Load scene

Load Scene, as used in the earlier example, loads the Scene directly with loading taking place during the next frame

While the Scene loads, the game will freeze.

Music and audio will continue to play but the game will be unresponsive until the load has finished.

Depending on the amount of time that's required to load the next Scene, this pause can give the impression that the game has crashed, unless you hide the pause with a Loading Screen (more on that later).



Load Scene Async

Load Scene Async works in a similar way to **Load Scene**, except the loading takes place as a background operation and is spread over multiple frames:

```
SceneManager.LoadSceneAsync(sceneToLoad);
```

There are a few reasons that you might want to do this:

For example, you might want to keep the current Scene running until the new Scene is ready. Or, if your loading screen includes animated elements, such as tooltips or a loading



LoadSceneMode

Use LoadSceneMode to choose what type of Scene loads when using [SceneManager.LoadScene](#). The available modes are Single and Additive.

Single mode loads a standard Unity Scene which then appears on its own in the Hierarchy window.

Additive loads a Scene which appears in the Hierarchy window while another is active.



Keeping game object between scenes

The load of a new [Scene](#) destroys all current [Scene](#) objects. Call [Object.DontDestroyOnLoad](#) to preserve an Object during scene loading. If the target Object is a component or [GameObject](#), Unity also preserves all of the [Transform](#)'s children. [Object.DontDestroyOnLoad](#) only works for root GameObjects or components on root GameObjects. [Object.DontDestroyOnLoad](#) does not return a value.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DontDestroy : MonoBehaviour
{
    void Awake()
    {
        GameObject[] objs =
GameObject.FindGameObjectsWithTag("music");

        if (objs.Length > 1)
        {
            Destroy(this.gameObject);
        }
    }
}
```

Build settings

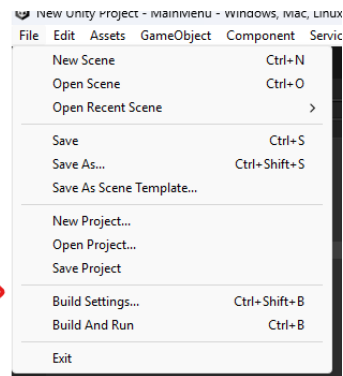
Adding a scene to the Build order



[14:09:28] Scene 'Level1' couldn't be loaded because it has not been added to the build settings or the AssetBundle has not been loaded.
To add a scene to the build settings use the menu File->Build Settings...

Use the **Scene List** to organize the scenes in your build. Unity builds scenes in the order they appear in the list. You can **add**, **exclude**, **remove**, and **reorder** scenes in the list.

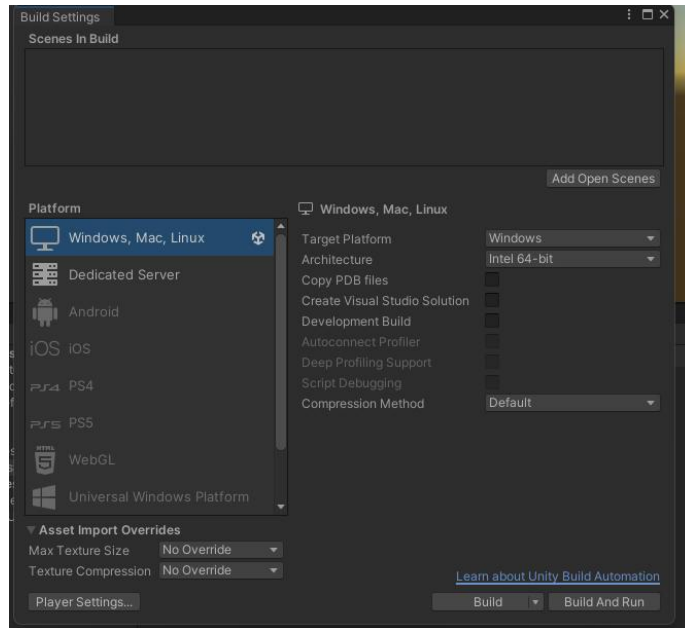
Access the scene list by navigating to **File > Build Settings**



Build settings

This window allows you to specify what scenes are part of your game and which are not.

It also allows you to create a standalone executable for your game.



How to add scene to the build settings

Simply drag and drop the scene you want in your game and drop them in the **Scene in Build** panel

